

TMC6460 Qt Actuator Control Workflow

Velocity-mode-only control, manual auto-calibration, saved hold-torque learning, and API call stack

Generated from the latest manual-calibration saved-file code package

This PDF explains the current Qt/TMC6460 actuator software architecture, the relevant APIs, the complete command flow, and the automatic end-to-end torque calibration workflow. The design keeps runtime motion in velocity mode only. Position mode and torque mode are disabled by macro. POSITION_ACTUAL is still read as feedback for end-stop detection, but the code does not command a position target.

- Connect + Initialize does not start the motor automatically.
- Start Calibration button starts the end-to-end calibration run.
- Calibration values are saved to calibration/tmc6460_calibration.ini.
- On restart, the saved learned hold torque/flux limit is loaded and used for hold/stall/E-stop.
- No stop path should set torque/flux limit or target to zero while powered.

1. Software Architecture

The code is organized into three logical layers. MainWindow owns the GUI and emits commands. MotorWorker owns actuator state, calibration, monitoring, stall/end logic, and safety hold. Tmc6460QtInterface owns serial communication and TMC6460 register-level APIs.

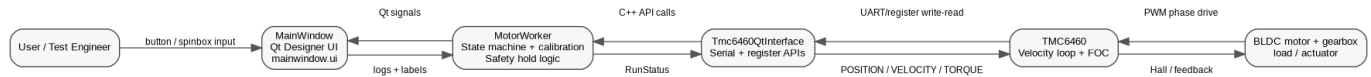


Figure 1 - Layered architecture and data flow

Layer	Main files	Responsibility
GUI layer	mainwindow.ui, mainwindow.cpp, mainwindow.h	Qt Designer UI, button/slider/spinbox handling, logs, display updates, signals to worker.
Control layer	MotorWorker.cpp, MotorWorker.h	Connection workflow, auto-calibration state machine, end-stop detection, learned hold torque, safe E-stop/hold.
TMC interface layer	Tmc6460QtInterface.cpp, Tmc6460QtInterface.h	Serial port, register read/write, TMC6460 configuration, velocity target, torque/flux limit, run-status reads.
Configuration	ActuatorConfig.h	Single location for enable/disable macros and default torque/velocity/calibration constants.

2. Key macros and current build behavior

The main project-level macros are centralized in ActuatorConfig.h to avoid duplicated constants. These macros describe the current build behavior.

Macro	Current value	Meaning
TMC6460_ENABLE_TORQUE_MODE	0	Torque command mode is disabled. The code should not run torque-target control.
TMC6460_ENABLE_POSITION_MODE	0	Position command mode is disabled. POSITION_ACTUAL is feedback only.
DEFAULT_VELOCITY_TORQUE_LIMIT_RAW	3000	Default torque/flux limit used when no saved calibration exists.
MIN_VELOCITY_TORQUE_LIMIT_RAW	300	Minimum non-zero torque/flux limit. Prevents unsafe zero-limit hold.
MAX_VELOCITY_TORQUE_LIMIT_RAW	3000	Upper clamp for learned/default hold limit.
DEFAULT_AUTO_TORQUE_LEARN_VELOCITY_RAW	4000000	Auto-calibration velocity magnitude. First leg uses +4000000, second leg uses -4000000.
TMC6460_ENABLE_AUTO_TORQUE_LEARN_ON_INIT	0	Connect + Initialize does not start calibration.
TMC6460_ENABLE_AUTO_END_TO_END_CALIBRATION	1	The Start Calibration button can run auto end-to-end calibration.
AUTO_TORQUE_HOLD_MARGIN_RAW	300	Margin added to measured torque to estimate hold torque/flux limit.
TMC6460_ENABLE_FINAL_END_CREEP	1	After possible end detection, perform final creep confirmation before holding.

```
// ActuatorConfig.h
```

```
#define TMC6460_ENABLE_TORQUE_MODE
```

```
0
```

```
#define TMC6460_ENABLE_POSITION_MODE          0
#define DEFAULT_VELOCITY_TORQUE_LIMIT_RAW      3000
#define MIN_VELOCITY_TORQUE_LIMIT_RAW          300
#define MAX_VELOCITY_TORQUE_LIMIT_RAW          3000
#define DEFAULT_AUTO_TORQUE_LEARN_VELOCITY_RAW 4000000
#define TMC6460_ENABLE_AUTO_TORQUE_LEARN_ON_INIT 0
#define TMC6460_ENABLE_AUTO_END_TO_END_CALIBRATION 1
#define TMC6460_CALIBRATION_FILE_NAME          "tmc6460_calibration.ini"
#define AUTO_TORQUE_HOLD_MARGIN_RAW            300
#define TMC6460_USE_VELOCITY_ZERO_HOLD_AFTER_END 1
#define TMC6460_ENABLE_ACTUATOR_STOP_DETECTION 1
#define TMC6460_ENABLE_FINAL_END_CREEP         1
```

3. Connect + Initialize workflow

Connect + Initialize only opens the serial port, configures the TMC6460, reads the chip ID, and loads any saved calibration file. It intentionally does not start motion. This is controlled by `TMC6460_ENABLE_AUTO_TORQUE_LEARN_ON_INIT = 0`.

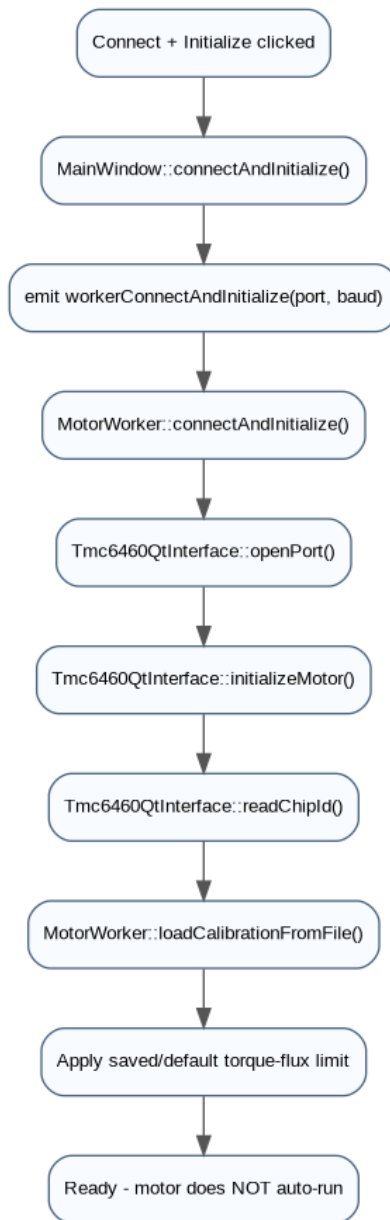


Figure 2 - Connect + Initialize call stack

API call stack

MainWindow::connectAndInitialize()

- > emit workerConnectAndInitialize(portName, baudRate)
- > MotorWorker::connectAndInitialize(portName, baudRate)
 - > ensureInterface()
 - > resetVelocityRampState()
 - > stopCalibrationEndMonitor(true)
 - > resetAutoCalibrationState()
 - > Tmc6460QtInterface::openPort(portName, baudRate)
 - > Tmc6460QtInterface::initializeMotor()
 - > Tmc6460QtInterface::readChipId(&chipId)
 - > MotorWorker::loadCalibrationFromFile()
 - > emit connected(true, chipId, "Connected and initialized")

Important result

- The actuator remains stopped after Connect + Initialize.
- If calibration/tmc6460_calibration.ini exists and is valid, learnedHoldTorqueFluxLimitRaw is loaded.
- If no valid file exists, learnedHoldTorqueFluxLimitRaw is set to DEFAULT_VELOCITY_TORQUE_LIMIT_RAW.
- The loaded/default hold value is immediately applied as a non-zero torque/flux limit.

4. Start Calibration button workflow

Calibration now starts only when the user clicks Start Calibration in the Connection / Safety area. This is the manual entry point for the automatic end-to-end torque learning sequence.

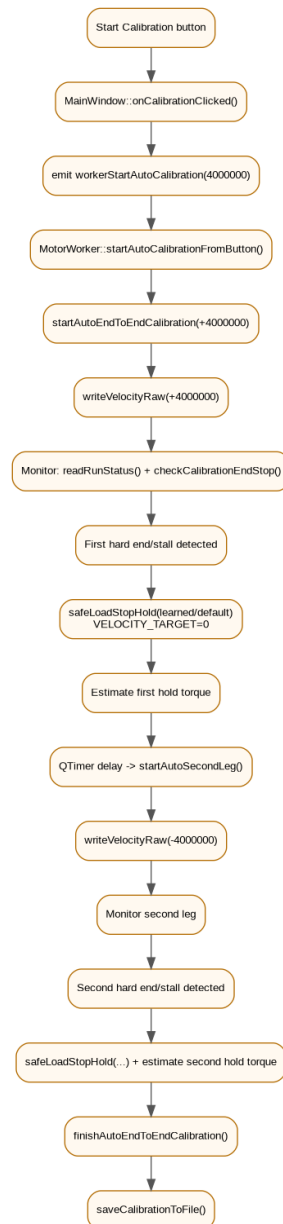


Figure 3 - Manual auto-calibration workflow

Button call stack

MainWindow::onCalibrationClicked()

-> calibrationVelocityRaw = DEFAULT_AUTO_TORQUE_LEARN_VELOCITY_RAW

-> emit workerStartAutoCalibration(calibrationVelocityRaw)

```
-> MotorWorker::startAutoCalibrationFromButton(rawVelocity)
-> verify serial port is open
-> startAutoEndToEndCalibration(rawVelocity)
```

First leg setup

```
MotorWorker::startAutoEndToEndCalibration(firstVelocityRaw)
-> limitedFirstVelocity = clamp(firstVelocityRaw)
-> autoCalibrationActive = true
-> autoCalibrationState = FirstLeg
-> autoCalibrationFirstVelocityRaw = +4000000
-> autoCalibrationSecondVelocityRaw = -4000000
-> readRunStatus(&before)
-> record first start position and start torque
-> writeVelocityRaw(+4000000)
-> startCalibrationEndMonitor(+4000000)
```

The first leg is treated as the release/down candidate in logs. The second leg is treated as lift/return candidate. The names are diagnostic labels; the actual physical meaning depends on your actuator orientation and sign convention.

5. Runtime monitoring and end-stop detection

During calibration and velocity runs, MotorWorker periodically reads TMC6460 feedback and decides whether the actuator is still moving or has reached a hard end/stall. The controller remains in velocity mode; POSITION_ACTUAL is used only as feedback.

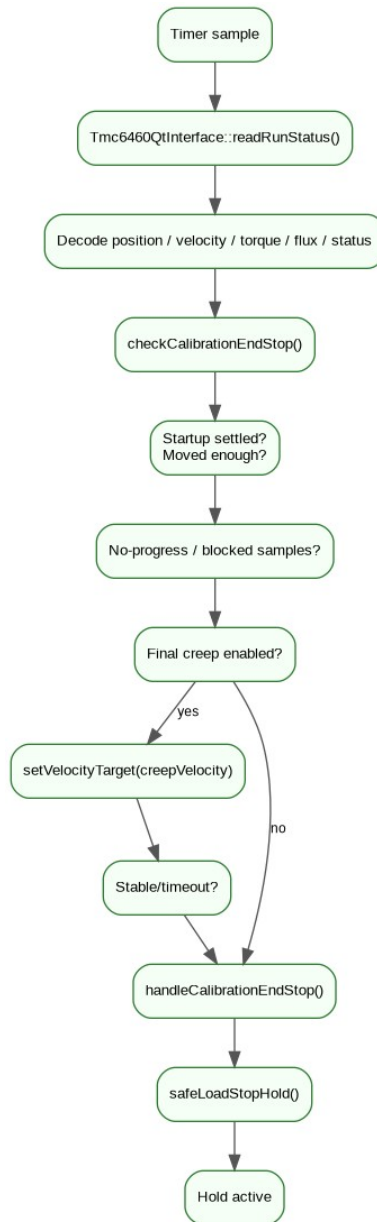


Figure 4 - End-stop and final-creep detection

Monitor call stack

```

MotorWorker::readStatus()
-> Tmc6460QtInterface::readRunStatus(&status)
-> emit statusReady(status)
-> if calibrationMonitorActive:
    checkCalibrationEndStop(status)
  
```

Feedback used

Feedback field	Source / register concept	Use
positionActual / positionActualSigned	POSITION_ACTUAL	Detect direction, travel counts, no-progress condition, end-stop position.
velocityActualRaw	VELOCITY_ACTUAL	Detect velocity collapse at hard stop or stall.
torqueActualRaw	TORQUE_FLUX_ACTUAL lower/decoded torque	Estimate moving torque and hold torque requirement.
fluxActualRaw	TORQUE_FLUX_ACTUAL	Diagnostic feedback; logged for

	upper/decoded flux	tuning.
chipStatusFlags	CHIP_STATUS_FLAGS	Fault/status logging.
chipEvents	CHIP_EVENTS	Fault/event logging.

End detection logic overview

- Startup ignore: initial acceleration is ignored for a minimum run time to avoid false end detection.
- Minimum travel: the actuator must move enough counts before a hard-end classification is accepted.
- No-progress check: position delta becomes very small for several blocked samples.
- Velocity/load check: velocity collapses and/or torque is loaded.
- Final creep: if enabled, the motor continues slowly in the same direction to confirm the hard mechanical end.
- Safety timeout: if motion does not complete in time, the code stops and holds rather than running indefinitely.

6. What happens when an end or stall is detected

When the first or second end is detected, the stop path must not remove torque. The code uses `safeLoadStopHold()`, which keeps velocity mode active, keeps the driver enabled, writes a non-zero torque/flux limit, and sets `VELOCITY_TARGET` to 0.

End/stall handling call stack

```
MotorWorker::checkCalibrationEndStop(status)
-> handleCalibrationEndStop(reason, status, progressCounts)
-> estimateHoldTorqueLimitFromStatus(status, movingMaxAbsRaw)
-> tmc->safeLoadStopHold(learnedHoldTorqueFluxLimitRaw)
-> tmc->readRunStatus(&settled)
-> update first/second end position and hold torque estimate
-> if FirstLeg: QTimer::singleShot(..., startAutoSecondLeg)
-> if SecondLeg: finishAutoEndToEndCalibration()
```

safeLoadStopHold() behavior

```
Tmc6460QtInterface::safeLoadStopHold(holdTorqueFluxLimitRaw)
-> clamp holdTorqueFluxLimitRaw to non-zero min/max
-> write FOC_PID_TORQUE_FLUX_LIMITS = (limit << 16) | limit
-> write MCC_CONFIG_MOTOR_MOTION = VELOCITY mode
-> write FOC_PID_VELOCITY_TARGET = 0
-> driver remains enabled
```

This is a powered hold. It can hold only while the 24 V supply, TMC6460, gate driver, and motor phases are active. It is not a power-failure safety mechanism.

7. Auto-calibration torque learning

Auto-calibration estimates a safe hold torque/flux limit from measured torque values during the first and second legs. The code records start torque, maximum moving torque, end torque, and settled hold torque. The larger estimate from both ends becomes the learned hold limit.

Torque estimate algorithm

```
qint16 MotorWorker::estimateHoldTorqueLimitFromStatus(status, movingMaxAbsRaw)
{
    measuredAbs = max(abs(status.torqueActualRaw), abs(movingMaxAbsRaw));
    return clampHoldTorqueLimit(measuredAbs + AUTO_TORQUE_HOLD_MARGIN_RAW);
}
```


Clamp behavior

```
qint16 MotorWorker::clampHoldTorqueLimit(qint32 raw) const
{
    return qBound(MIN_VELOCITY_TORQUE_LIMIT_RAW,
        abs(raw),
        MAX_VELOCITY_TORQUE_LIMIT_RAW);
}
```

Calibration result selection

```
MotorWorker::finishAutoEndToEndCalibration()
-> endToEndCounts = abs(secondEndPosition - firstEndPosition)
-> learnedHoldTorqueFluxLimitRaw = clamp(max(firstHold, secondHold))
-> tmc->setVelocityTorqueFluxLimit(learnedHoldTorqueFluxLimitRaw)
-> tmc->safeLoadStopHold(learnedHoldTorqueFluxLimitRaw)
-> saveCalibrationToFile(endToEndCounts)
```

Because first and second directions can have very different load conditions, the learned value is the larger of the two required hold estimates. This learned value is then used for later end/stall/E-stop hold conditions.

8. Calibration file save and reload

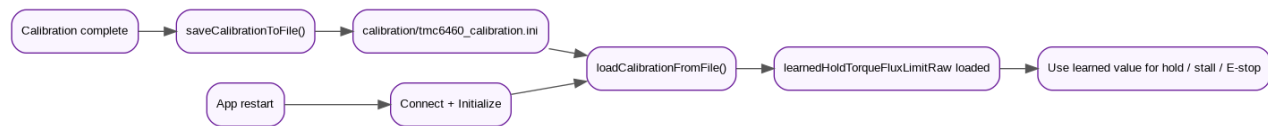


Figure 5 - Saved calibration file flow

The calibration file is stored beside the application under a calibration folder. It is written with QSettings in INI format. On the next app start, Connect + Initialize loads the file and applies the saved learned hold torque/flux limit.

File path

```
<application_folder>/calibration/tmc6460_calibration.ini
```

Save API

```
MotorWorker::saveCalibrationToFile(endToEndCounts)
-> settings.setValue("Calibration/valid", true)
-> settings.setValue("Calibration/timestamp", currentDateTime)
-> settings.setValue("Calibration/velocityRaw", abs(firstVelocityRaw))
-> settings.setValue("Calibration/firstEndPosition", firstEndPosition)
-> settings.setValue("Calibration/secondEndPosition", secondEndPosition)
-> settings.setValue("Calibration/firstTravelCounts", firstTravelCounts)
-> settings.setValue("Calibration/secondTravelCounts", secondTravelCounts)
-> settings.setValue("Calibration/endToEndCounts", endToEndCounts)
-> settings.setValue("Calibration/firstHoldTorqueFluxLimit", firstHold)
-> settings.setValue("Calibration/secondHoldTorqueFluxLimit", secondHold)
-> settings.setValue("Calibration/learnedHoldTorqueFluxLimit", learnedHold)
-> settings.sync()
```

Load API

```
MotorWorker::loadCalibrationFromFile()
-> path = calibrationFilePath()
```

```

-> if Calibration/valid is false:
    learnedHoldTorqueFluxLimitRaw = DEFAULT_VELOCITY_TORQUE_LIMIT_RAW
    tmc->setVelocityTorqueFluxLimit(learnedHoldTorqueFluxLimitRaw)
    return
-> read first/second end positions
-> read travel counts
-> read first/second hold torque values
-> learnedHoldTorqueFluxLimitRaw = clamp(saved learned value)
-> tmc->setVelocityTorqueFluxLimit(learnedHoldTorqueFluxLimitRaw)

```

Example INI content

```

[Calibration]
valid=true
timestamp=2026-06-29T20:15:00
velocityRaw=4000000
firstEndPosition=12345678
secondEndPosition=12301234
firstTravelCounts=44444
secondTravelCounts=44510
endToEndCounts=44476
firstHoldTorqueFluxLimit=900
secondHoldTorqueFluxLimit=1600
learnedHoldTorqueFluxLimit=1600
torqueModeEnabled=0
positionModeEnabled=0

```

9. Normal operation after calibration

After calibration, normal CW/CCW or direct velocity commands continue to use the existing velocity mode core. The learned hold value is available for future hold, stall, and E-stop conditions.

CW/CCW command stack

```

MainWindow::onVelocityCwClicked()
-> speed = abs(ui->velocityDirectSpin->value())
-> emit workerApplyVelocityRaw(+speed)
-> MotorWorker::applyVelocityRaw(+speed)
-> writeVelocityRaw(+speed)
-> Tmc6460QtInterface::setVelocityTarget(+speed)
-> write VELOCITY mode
-> write FOC_PID_VELOCITY_TARGET = +speed
-> startCalibrationEndMonitor(+speed) if stop detection is enabled

MainWindow::onVelocityCcwClicked()
-> speed = abs(ui->velocityDirectSpin->value())
-> emit workerApplyVelocityRaw(-speed)
-> MotorWorker::applyVelocityRaw(-speed)
-> writeVelocityRaw(-speed)
-> monitor end/stall feedback

```

Velocity zero command

Velocity = 0 is treated as active hold, not idle. It should call `safeLoadStopHold()` and should not write torque/flux target = 0 or torque/flux limit = 0.

```
Tmc6460QtInterface::setVelocityTarget(0)
-> safeLoadStopHold(currentVelocityTorqueFluxLimitRaw)
-> torque/flux limit remains non-zero
-> motion mode remains VELOCITY
-> VELOCITY_TARGET = 0
-> driver remains enabled
```

10. Relevant public APIs

MainWindow UI slots and signals

API	Direction	Purpose
connectAndInitialize()	UI slot	Starts connection and initialization only. Does not auto-run calibration.
onCalibrationClicked()	UI slot	Starts manual auto-calibration by emitting workerStartAutoCalibration(4000000).
onVelocityCwClicked() / onVelocityCcwClicked()	UI slot	Converts direct speed to signed velocity command.
onEstopClicked()	UI slot	Emits workerEmergencyStop; worker applies safe hold.
workerConnectAndInitialize(port, baud)	signal	Queued command to MotorWorker.
workerStartAutoCalibration(rawVelocity)	signal	Queued command to start calibration.
workerApplyVelocityRaw(rawVelocity)	signal	Queued velocity command.
workerEmergencyStop()	signal	Queued safe E-stop/hold command.

MotorWorker APIs

API	Purpose
connectAndInitialize(portName, baudRate)	Open serial, initialize TMC6460, read chip ID, load calibration file.
startAutoCalibrationFromButton(rawVelocity)	Validate connection and start auto end-to-end calibration.
startAutoEndToEndCalibration(firstVelocityRaw)	Initialize FirstLeg state, take start snapshot, run +velocity, start monitor.
startAutoSecondLeg()	Reverse direction after first end, take second start snapshot, run -velocity, start monitor.
checkCalibrationEndStop(status)	Analyze feedback for hard end/stall/no-progress/final creep.
handleCalibrationEndStop(reason, status, progressCounts)	Stop/hold safely, record end position and torque, transition to reverse or finish.
finishAutoEndToEndCalibration()	Compute learned hold value, apply it, save INI file.
loadCalibrationFromFile()	Load saved calibration and apply learned/default torque/flux limit.
saveCalibrationToFile(endToEndCounts)	Persist calibration results.
emergencyStop()	Cancel motion/calibration and apply safe powered hold.

Tmc6460QtInterface APIs

API	Register-level intent
openPort(portName, baudRate)	Open QSerialPort to the Arduino/TMC bridge.
initializeMotor()	Write the TMC6460 working configuration.
readChipId(&chipId)	Read chip ID for connection proof.
setVelocityTarget(rawVelocity)	Set velocity mode and FOC_PID_VELOCITY_TARGET; zero goes to safeLoadStopHold.
setVelocityTorqueFluxLimit(rawLimit)	Write FOC_PID_TORQUE_FLUX_LIMITS with torque=flux=rawLimit.

safeLoadStopHold(rawLimit)	Apply non-zero torque/flux limit, velocity mode, VELOCITY_TARGET=0.
readRunStatus(&status)	Read/decode position, velocity, torque/flux actual, status flags, events.
emergencyStop()	Calls safeLoadStopHold(current/learned non-zero limit).

11. Register concepts used by the workflow

Register concept	Used by	Purpose
MCC_CONFIG_MOTOR_MOTION	setVelocityTarget(), safeLoadStopHold()	Keeps controller in velocity mode for motion and powered hold.
FOC_PID_VELOCITY_TARGET	setVelocityTarget(), safeLoadStopHold()	Positive/negative raw velocity for motion; 0 for powered hold.
FOC_PID_TORQUE_FLUX_LIMITS	setVelocityTorqueFluxLimit(), safeLoadStopHold()	Sets torque and flux limits. Code uses same non-zero value for both.
POSITION_ACTUAL	readRunStatus(), checkCalibrationEndStop()	Feedback only. Used for travel/no-progress/end position.
VELOCITY_ACTUAL	readRunStatus(), checkCalibrationEndStop()	Detects actual speed and speed collapse.
TORQUE_FLUX_ACTUAL	readRunStatus(), torque learning	Decoded into torqueActualRaw and fluxActualRaw for logging and learned limit.
CHIP_STATUS_FLAGS / CHIP_EVENTS	readRunStatus()	Fault/event debug logging.

12. Safety invariants

The following invariants are intentionally part of the code design because the actuator can be back-driven by load if powered holding torque is removed.

- Do not write FOC_PID_TORQUE_FLUX_LIMITS = 0 during powered operation.
- Do not write FOC_PID_TORQUE_FLUX_TARGET = 0 as a stop/hold mechanism for a loaded actuator.
- Do not switch to idle/PWM freewheel when a load is attached.
- Do not disable the gate driver during load hold.
- Velocity target 0 means active velocity-zero hold, not motor disable.
- E-stop in this software is safe hold, not driver disable.
- Powered hold is not fail-safe during power loss. A mechanical brake/latch/support is required for a 10 kg load.

Correct stop/hold API sequence

```
safeLoadStopHold(limitRaw)
-> clamp limitRaw to MIN/MAX
-> setVelocityTorqueFluxLimit(limitRaw)
-> set motion mode = VELOCITY
-> set VELOCITY_TARGET = 0
-> keep driver enabled
```

Unsafe stop/hold sequence - must not be used with load

```
// Unsafe with load:
setVelocityTorqueFluxLimit(0)
setTorqueFluxTarget(0)
switch to idle / PWM_ON idle
disable gate driver
power off without mechanical brake/support
```

13. Logs to verify during testing

Log prefix	Meaning
CAL_FILE_LOAD	Calibration file was loaded or default values were used.
CALIBRATION_BUTTON	Start Calibration button requested auto-calibration.
AUTO_TORQUE_LEARN_BEGIN	First leg started. Shows first and second velocity command values.
AUTO_TORQUE_FIRST_START_SNAPSHOT	Start feedback snapshot for first leg.
CAL_MONITOR_SAMPLE	Periodic position/velocity/torque sample.
FINAL_END_CREEP_START	Possible hard end detected; slow creep confirmation started.
CAL_END_HARD_MECHANICAL_END	Hard end/stall detected.
CAL_SETTLED_END	Snapshot after velocity-zero hold is applied.
AUTO_TORQUE_FIRST_HOLD_ESTIMATE	Required hold estimate for first leg.
AUTO_CAL_SECOND_LEG_START	Reverse leg started.
AUTO_TORQUE_SECOND_HOLD_ESTIMATE	Required hold estimate for second leg.
AUTO_TORQUE_LEARN_COMPLETE	Both ends found; calibration is complete.
CAL_FILE_SAVE	INI file saved with learned hold value.
SAFE_LOAD_STOP_HOLD	Powered hold applied: non-zero limit + velocity target 0.

14. Practical test sequence

1. Mechanically support the 10 kg load before starting any powered test.
2. Open the application and click Connect + Initialize. Confirm chip ID and CAL_FILE_LOAD log.
3. Click Start Calibration only when it is safe for the actuator to move end-to-end.
4. Observe first leg run to one end, hold, and log AUTO_TORQUE_FIRST_HOLD_ESTIMATE.
5. Observe automatic reverse to the opposite end, hold, and log AUTO_TORQUE_SECOND_HOLD_ESTIMATE.
6. Confirm AUTO_TORQUE_LEARN_COMPLETE and CAL_FILE_SAVE logs.
7. Restart the application and click Connect + Initialize. Confirm that the same learnedHoldTorqueFluxLimit is loaded.
8. Use CW/CCW and E-stop. Confirm all stop paths log SAFE_LOAD_STOP_HOLD or equivalent active hold behavior.

15. What is not included intentionally

- No position target control. Position mode is disabled by TMC6460_ENABLE_POSITION_MODE = 0.
- No torque target control. Torque mode is disabled by TMC6460_ENABLE_TORQUE_MODE = 0.
- No automatic motion after Connect + Initialize. Calibration is button-driven.
- No power-fail safety. Add normally locked brake, mechanical latch, self-locking mechanism, or external support.

16. Quick reference: major call stacks

Connection

```
MainWindow::connectAndInitialize
-> workerConnectAndInitialize
-> MotorWorker::connectAndInitialize
-> Tmc6460QtInterface::openPort
-> Tmc6460QtInterface::initializeMotor
-> Tmc6460QtInterface::readChipId
-> MotorWorker::loadCalibrationFromFile
```

Manual calibration

```
MainWindow::onCalibrationClicked
-> workerStartAutoCalibration(4000000)
```

```
-> MotorWorker::startAutoCalibrationFromButton  
-> MotorWorker::startAutoEndToEndCalibration  
-> MotorWorker::writeVelocityRaw(+4000000)  
-> Tmc6460QtInterface::setVelocityTarget(+4000000)
```

End detected

```
MotorWorker::readStatus  
-> Tmc6460QtInterface::readRunStatus  
-> MotorWorker::checkCalibrationEndStop  
-> MotorWorker::handleCalibrationEndStop  
-> Tmc6460QtInterface::safeLoadStopHold  
-> MotorWorker::estimateHoldTorqueLimitFromStatus  
-> MotorWorker::startAutoSecondLeg OR finishAutoEndToEndCalibration
```

Save and reuse learned hold

```
MotorWorker::finishAutoEndToEndCalibration  
-> MotorWorker::saveCalibrationToFile
```

Next restart:

```
MotorWorker::connectAndInitialize  
-> MotorWorker::loadCalibrationFromFile  
-> Tmc6460QtInterface::setVelocityTorqueFluxLimit(learnedHoldTorqueFluxLimitRaw)
```